

■ LEVEL 15 • BINARY SEARCH TREE (BST)

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Left < root < right. Ordering lets search discard one side; inorder traversal is sorted.
Reference Examples	Search BST • Validate BST • Kth Smallest • Lowest Common Ancestor • Predecessor/Successor
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 15 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Search in a Binary Search Tree	BST Search	#700
2	Validate Binary Search Tree	Inorder / Bounds	#98
3	Kth Smallest Element in a BST	Inorder Traversal	#230
4	Lowest Common Ancestor of a BST	BST Property	#235
5	Convert Sorted Array to BST	Divide and Conquer	#108
6	Inorder Successor in BST	BST Search	Absent
7	Search in a Binary Search Tree	BST Search	700
8	Validate Binary Search Tree	BST Inorder	98
9	Kth Smallest Element in a BST	Inorder Traversal	230
10	Lowest Common Ancestor of a BST	BST Ordering	235

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Search in a Binary Search Tree

Pattern: BST Search

LeetCode: #700

■ Input	BST root and target
■ Output	node containing target, or null
■ Constraints	BST property
■ Main Idea / Pattern	Compare target with root and move only into the possible side.
■■ Time Complexity	O(h)
■ Space Complexity	O(1) iterative

■ Optimal C++ Code

```
TreeNode* searchBST(TreeNode* root, int val) {  
    while (root) {  
        if (root->val == val) return root;  
        if (val < root->val)  
            root = root->left;  
        else  
            root = root->right;  
    }  
    return nullptr;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Validate Binary Search Tree

Pattern: Inorder / Bounds

LeetCode: #98

■ Input	binary tree root
■ Output	whether it is a valid BST
■ Constraints	node values fit integer range
■ Main Idea / Pattern	Every node must lie within a valid lower and upper bound.
■■ Time Complexity	O(n)
■ Space Complexity	O(h)

■ Optimal C++ Code

```
bool valid(TreeNode* root, long long low, long long high) {
    if (!root) return true;

    if (root->val <= low || root->val >= high)
        return false;

    return valid(root->left, low, root->val) &&
           valid(root->right, root->val, high);
}

bool isValidBST(TreeNode* root) {
    return valid(root, LLONG_MIN, LLONG_MAX);
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Kth Smallest Element in a BST

Pattern: Inorder Traversal

LeetCode: #230

■ Input	BST root and k
■ Output	kth smallest value
■ Constraints	$1 \leq k \leq$ number of nodes
■ Main Idea / Pattern	BST inorder traversal is sorted. Stop when k reaches zero.
■ Time Complexity	$O(h + k)$
■ Space Complexity	$O(h)$

■ Optimal C++ Code

```
int kthSmallest(TreeNode* root, int k) {
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) {
            st.push(cur);
            cur = cur->left;
        }
        cur = st.top();
        st.pop();
        if (--k == 0)
            return cur->val;
        cur = cur->right;
    }
    return -1;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Lowest Common Ancestor of a BST

Pattern: BST Property

LeetCode: #235

■ Input	BST root and two nodes
■ Output	lowest common ancestor
■ Constraints	p and q exist
■ Main Idea / Pattern	If both values are smaller go left; if both larger go right; otherwise current node is the split point.
■ Time Complexity	O(h)
■ Space Complexity	O(1)

■ Optimal C++ Code

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {  
    while (root) {  
        if (p->val < root->val && q->val < root->val)  
            root = root->left;  
        else if (p->val > root->val && q->val > root->val)  
            root = root->right;  
        else  
            return root;  
    }  
    return nullptr;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Convert Sorted Array to BST

Pattern: Divide and Conquer

LeetCode: #108

■ Input	sorted array
■ Output	height-balanced BST
■ Constraints	array sorted ascending
■ Main Idea / Pattern	Middle element becomes root; recursively build left and right halves.
■■ Time Complexity	O(n)
■ Space Complexity	O(log n)

■ Optimal C++ Code

```
TreeNode* build(vector<int>& nums, int l, int r) {
    if (l > r) return nullptr;

    int mid = l + (r - l) / 2;
    TreeNode* root = new TreeNode(nums[mid]);

    root->left = build(nums, l, mid - 1);
    root->right = build(nums, mid + 1, r);

    return root;
}

TreeNode* sortedArrayToBST(vector<int>& nums) {
    return build(nums, 0, nums.size() - 1);
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Inorder Successor in BST

Pattern: BST Search

LeetCode: ■ Absent

■ Input	BST root and node p
■ Output	smallest node strictly greater than p
■ Constraints	valid BST
■ Main Idea / Pattern	When root is greater than p, it is a candidate and move left; otherwise move right.
■■ Time Complexity	O(h)
■ Space Complexity	O(1)

■ Optimal C++ Code

```
TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {  
    TreeNode* ans = nullptr;  
    while (root) {  
        if (root->val > p->val) {  
            ans = root;  
            root = root->left;  
        } else {  
            root = root->right;  
        }  
    }  
    return ans;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Search in a Binary Search Tree

Pattern: BST Search

LeetCode: 700

Input	root, val
Output	node containing val
Constraints	1 <= nodes <= 1e4
Main Idea / Pattern	Use the BST ordering to choose only the side that can contain the value.
Time Complexity	O(h)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    TreeNode* searchBST(TreeNode* root, int val) {
        while (root) {
            if (root->val == val) return root;
            if (val < root->val) root = root->left;
            else root = root->right;
        }
        return nullptr;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Validate Binary Search Tree

Pattern: BST Inorder

LeetCode: 98

Input	root
Output	whether tree satisfies strict BST ordering
Constraints	1 <= nodes <= 1e4
Main Idea / Pattern	Inorder traversal of a valid BST is strictly increasing.
Time Complexity	O(n)
Space Complexity	O(h)

Optimal C++ Code

```
class Solution {
    long long prev = LLONG_MIN;
public:
    bool isValidBST(TreeNode* root) {
        if (!root) return true;
        if (!isValidBST(root->left)) return false;
        if (root->val <= prev) return false;
        prev = root->val;
        return isValidBST(root->right);
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Kth Smallest Element in a BST

Pattern: Inorder Traversal

LeetCode: 230

Input	root, k
Output	k-th smallest value
Constraints	$1 \leq k \leq \text{nodes} \leq 1e4$
Main Idea / Pattern	Inorder visits BST values in sorted order; stop at the k-th node.
Time Complexity	$O(h + k)$
Space Complexity	$O(h)$

Optimal C++ Code

```
class Solution {
    int k;
    int ans;
    void inorder(TreeNode* root) {
        if (!root || k == 0) return;
        inorder(root->left);
        if (--k == 0) {
            ans = root->val;
            return;
        }
        inorder(root->right);
    }
public:
    int kthSmallest(TreeNode* root, int k) {
        this->k = k;
        inorder(root);
        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Lowest Common Ancestor of a BST

Pattern: BST Ordering

LeetCode: 235

Input	root, p, q
Output	lowest common ancestor
Constraints	2 <= nodes <= 1e4
Main Idea / Pattern	If both targets are smaller go left; if both larger go right; otherwise current node is the split point.
Time Complexity	O(h)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        while (root) {
            if (p->val < root->val && q->val < root->val) {
                root = root->left;
            } else if (p->val > root->val && q->val > root->val) {
                root = root->right;
            } else {
                return root;
            }
        }
        return nullptr;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.